settings.py: what matters and how to explain it

Teaching guide for the book catalog project (Django, Postgres, Nginx in Docker). Covers the core settings, static files, media files, logging and overriding the default ID field.

1. How to frame it

Say this first: **settings.py is the control panel of the project.** Django reads it once at startup, and every other part (ORM, templates, admin, middleware) asks it questions. It is plain Python, so it can read environment variables, which is exactly what we need in Docker: the same image runs in development and production, and only the values in `.env` change.

- Anything that differs between machines (secrets, hosts, DB credentials, debug flag) comes from the environment.
- Anything that describes the app itself (installed apps, middleware, templates) stays in the file.
- Reading with `os.environ["X"]` is deliberate: a missing variable crashes at startup instead of silently running with a wrong value.

2. The settings to know, in the order to teach them

Setting	What it does	How to say it
BASE_DIR	Absolute path of the project root, built with pathlib.	Never hardcode paths. Everything else is built from this.
SECRET_KEY	Signs sessions, CSRF tokens, password reset links.	If it leaks, anyone can forge sessions. Never commit it, read it from the environment.
DEBUG	Detailed error pages, static serving in runserver, no host check strictness.	A debug page shows settings and code to the visitor. It must be off outside your laptop.
ALLOWED_HOSTS	Host headers Django will answer to (when DEBUG is off).	Defence against forged Host headers. Wrong value gives a 400 error, a classic first Docker mistake.
CSRF_TRUSTED_ORIGINS	Origins allowed to POST when behind a proxy or another domain.	Must include the scheme, for example https://example.com. Needed once Nginx and TLS are in front.
INSTALLED_APPS	Apps Django loads: models, templates, admin, static, commands.	If an app is not listed it does not exist for Django. Forgetting it is the top cause of 'template not found' and missing migrations.
MIDDLEWARE	Ordered chain that wraps every request and response.	Order matters: it is a pipeline. Request goes down the list, response goes back up.
ROOT_URLCONF	The module where URL resolution starts.	This is the file where we first put our def views, before moving to include().
TEMPLATES	Template engine config: DIRS and APP_DIRS.	DIRS is the project-level folder (base.html lives here), APP_DIRS lets each app ship its own templates.
DATABASES	Connection to Postgres.	Host is the compose service name (db), not localhost. Containers talk by service name.
LANGUAGE_CODE, TIME_ZONE, USE_TZ	Locale and time handling.	Keep USE_TZ = True: store UTC in the DB, convert on display.
DEFAULT_AUTO_FIELD	Type of the automatic primary key.	See section 6.

A realistic core, environment driven (variable names match the .env.example from the Docker setup):

```
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
DEBUG = os.environ.get("DJANGO_DEBUG", "0") == "1"
ALLOWED_HOSTS = [h for h in os.environ.get("DJANGO_ALLOWED_HOSTS", "").split(",") if h]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["POSTGRES_DB"],
        "USER": os.environ["POSTGRES_USER"],
        "PASSWORD": os.environ["POSTGRES_PASSWORD"],
        "HOST": os.environ.get("POSTGRES_HOST", "db"),
        "PORT": os.environ.get("POSTGRES_PORT", "5432"),
    }
}
```

Gotcha to show live: environment variables are always strings. `DEBUG = os.environ.get('DEBUG')` is truthy even when the value is the string "0" or "False". That is why we compare with == "1".

3. Static files

Static files are the files you ship with the code: CSS, JavaScript, logos, the admin's own assets. They are the same for every visitor and are not changed at runtime.

```
STATIC_URL = "/static/"                      # URL prefix in the browser
STATIC_ROOT = BASE_DIR / "staticfiles"       # where collectstatic copies everything
STATICFILES_DIRS = [BASE_DIR / "static"]     # optional: project-wide assets folder
```

Three names, three jobs. Explain them as a story:

- **STATICFILES_DIRS** and each app's `static/` folder are the *sources*.
- **collectstatic** copies all sources into **STATIC_ROOT**, one flat folder.
- **STATIC_URL** is the address the browser uses. In our stack Nginx maps /static/ to that folder.

In Docker this is the shared volume: the entrypoint runs collectstatic in the web container, the named volume carries the files, and Nginx serves them read-only without touching Django.

Break it on purpose: set DEBUG to 0 with runserver and reload the admin. The CSS disappears, because runserver only serves static files in debug mode. This one demo explains why Nginx and collectstatic exist.

4. Media files

Media files are uploaded by users at runtime: book covers, author photos, attachments. They are data, not code, so they must never live in the image or in version control.

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Model side (needs Pillow installed):

```
cover = models.ImageField(upload_to="covers/%Y/%m/", blank=True)
```

Development only, add to the project urls.py so runserver can show uploads:

```
from django.conf import settings
from django.conf.urls.static import static

if settings.DEBUG:
```

```
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Production with the Docker setup needs three small additions, because the first compose file only serves statics:

```
# Dockerfile: create and own the folder, otherwise uploads fail with Permission denied
RUN mkdir -p /app/media && chown -R appuser:appuser /app/media

# docker-compose.yml
web:    volumes: [static_data:/app/staticfiles, media_data:/app/media]
nginx:  volumes: [..., media_data:/app/media:ro]
volumes: media_data:

# nginx/default.conf
location /media/ { alias /app/media/; }
```

Compare in one sentence: **static is what developers deploy, media is what visitors upload.** Static is rebuilt on every deploy, media must survive deploys, which is why it gets its own named volume (and a backup plan).

5. Logging

Explain the four pieces once and the dictionary stops looking scary:

- **Logger**: the thing your code calls (named, usually after the module).
- **Level**: DEBUG, INFO, WARNING, ERROR, CRITICAL. A logger only passes messages at or above its level.
- **Handler**: where the message goes (console, file, email).
- **Formatter**: how the line looks.

Container rule: log to the console (stdout) and let Docker collect it. Do not write log files inside the container. You read them with `docker compose logs -f web`, and gunicorn's access log already goes there thanks to `--access-logfile -`.

```
LOG_LEVEL = os.environ.get("DJANGO_LOG_LEVEL", "INFO")

LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "verbose": {"format": "{asctime} {levelname} {name} {message}", "style": "{"},
    },
    "handlers": {
        "console": {"class": "logging.StreamHandler", "formatter": "verbose"},
    },
    "root": {"handlers": ["console"], "level": LOG_LEVEL},
    "loggers": {
        "django": {"handlers": ["console"], "level": LOG_LEVEL, "propagate": False},
        "catalog": {"handlers": ["console"], "level": "DEBUG", "propagate": False},
        # development only: print every SQL query
        # "django.db.backends": {"handlers": ["console"], "level": "DEBUG"},
    },
}
```

Using it in a view (the logger name follows the module path, so it matches the 'catalog' entry above):

```
import logging
logger = logging.getLogger(__name__)

def book_detail(request, pk):
    logger.info("Book detail requested: pk=%s", pk)
```

Two points students get wrong: pass values as arguments (`logger.info('x %s', v)`) instead of f-strings, and never log passwords, tokens or full request bodies. `disable_existing_loggers:`

`False` is almost always what you want, otherwise third party loggers go silent.

6. Overriding the default ID field

Every model without an explicit primary key gets an automatic `id`. The *type* of that field is decided by **DEFAULT_AUTO_FIELD**. There are three levels, and the most specific one wins:

```
# 1. Project-wide default (settings.py)
DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"

# 2. Per app (catalog/apps.py)
class CatalogConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "catalog"

# 3. Per model: an explicit field named id (or any field with primary_key=True)
class Book(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
```

If neither level is set Django warns (models.W042) and falls back to a 32-bit AutoField, which is why startproject writes BigAutoField for you. To avoid repeating the override in every model, use an abstract base class:

```
import uuid
from django.db import models

class BaseModel(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True      # no table for this class, only inherited fields

class Genre(BaseModel):
    name = models.CharField(max_length=80, unique=True)
```

Which one to choose, and how to say it:

Option	Good for	Cost
BigAutoField (default)	Simple, fast, small indexes, readable URLs like /books/42/.	IDs are guessable and sequential. Fine unless the ID itself is sensitive.
UUIDField	Non-guessable URLs, merging data from several systems, generating IDs before the insert.	16 bytes, bigger indexes, random insert order. Ugly to type in the shell.
Natural key (for example ISBN)	Good for teaching what primary_key=True means.	Real-world values change and have exceptions. Avoid as a primary key in real projects.
BigAutoField plus a separate public_id UUID	Common compromise: fast integer joins inside, UUID in public URLs.	One extra unique indexed column.

Decide before the first migration. Changing the type of a primary key once tables hold data means migrating every foreign key that points to it. In the course schedule, make this choice at the start of Day 2, right before models.py is written. Suggested default for the catalog: BigAutoField, then UUID as a discussion or exercise.

7. Production checklist

Check	Why
DEBUG off, SECRET_KEY from the environment	No debug pages, no leaked signing key.

Check	Why
ALLOWED_HOSTS and CSRF_TRUSTED_ORIGINS set	Correct host handling behind Nginx. Origins need the scheme.
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")	Only when TLS terminates at Nginx, which already sends X-Forwarded-Proto. Tells Django the request was HTTPS.
SESSION_COOKIE_SECURE and CSRF_COOKIE_SECURE = True	Cookies only travel over HTTPS. Turn on together with TLS, not before.
python manage.py check --deploy	Django lists the remaining security warnings for you.
No secrets in git; .env in .dockerignore and .gitignore	The image must not contain credentials.

8. Classroom flow: break it on purpose (about 60 to 75 minutes)

For each block: open the setting, say the one-sentence explanation, then break it and let students read the error. Errors they have seen once are errors they recognise later.

Step	Break it by	What they learn
1. Environment	Remove POSTGRES_DB from .env and restart	Missing config fails loudly, KeyError at startup.
2. ALLOWED_HOSTS	Empty it with DEBUG off	The 400 Bad Request, and what the Host header is.
3. DEBUG	Trigger an error with DEBUG on, then off	What an attacker would see, and the plain 500 page.
4. Static	DEBUG off with runserver, reload admin	Why collectstatic and Nginx exist.
5. Media	Upload a cover before adding the volume, then after	Containers are disposable, volumes are not.
6. Logging	Add logger.info in a view, run docker compose logs -f web	Where output goes in a container.
7. ID field	Print Book._meta.pk and compare BigAutoField with UUID	Primary key type is a design decision with a cost.